UD1 — DESARROLLO DE UNA API REST CON SPRING BOOT

1. INTRODUCCIÓN Y EVOLUCIÓN DE LAS APLICACIONES

Evolución Histórica

- **Monolithic (1980s):** Aplicaciones de código único y centralizado.
- **Distributed monoliths (1990s):** Aplicaciones distribuidas mediante tecnologías propietarias y middleware (como CORBA o RMI en Java EE).
- **Internet-connected (2000s):** Mayor conectividad y uso de protocolos estándar.
- **Internet is the system (2010s):** Auge masivo de la web, microestándares (RFC) y protocolos sencillos como HTTP.
- **Intelligent connected (2020s):** Sistemas conectados e inteligentes actuales.

Desventajas de los modelos distribuidos antiguos (CORBA/RMI)

- Escalabilidad y evolución complejas.
- Dependencia de tecnologías propietarias.
- Problemas de comunicación, latencia y restricciones con cortafuegos (firewalls).

2. EL PROTOCOLO HTTP

Definición: Protocolo de transferencia de hipertexto enfocado en la arquitectura Cliente / Servidor y de naturaleza sin estado (stateless).

Estructura de los Mensajes

(Tanto en petición como en respuesta comparten una estructura similar):

- **Línea Inicial:** Método, ruta/URI y versión del protocolo (en peticiones) o versión, código de estado y mensaje (en respuestas).
- **Encabezados (Headers):** Metadatos en formato key: value que aportan contexto al mensaje.
- **Línea Vacía:** Delimitador obligatorio entre cabeceras y cuerpo.
- **Cuerpo del Mensaje (Body):** Contenido de los datos transmitidos.

Métodos (Verbos) HTTP Principales

- **GET:** Obtener o recuperar un recurso.
- **POST:** Crear un nuevo recurso.
- **PUT:** Modificar o actualizar un recurso existente.
- **DELETE:** Borrar un recurso.

Códigos de Respuesta HTTP

Se agrupan en rangos numéricos (1XX, 2XX, 3XX, 4XX, 5XX):

- **200 OK:** Operación exitosa.
- **201 Created:** Recurso creado exitosamente.
- **204 No Content:** Operación exitosa sin contenido de retorno (típico en borrados).

- **400 Bad Request:** Error de validación o petición incorrecta.
- **401 / 403:** Errores de autenticación y control de acceso.
- **404 Not Found:** Recurso no encontrado.
- **405 Method Not Allowed:** Método HTTP no permitido.
- **500+:** Errores internos del servidor.

3. ARQUITECTURA REST (REPRESENTATIONAL STATE TRANSFER)

Concepto: Estilo arquitectónico basado en un conjunto de principios para describir y acceder a recursos en la red utilizando las capacidades del protocolo HTTP.

Características principales

- Basado en HTTP.
- Arquitectura cliente-servidor.
- Sin estado (Stateless).
- Interfaz uniforme.
- Estructura en capas.

Identificación y Formatos

- **URIs orientadas a recursos:** Los endpoints se construyen usando sustantivos en plural y minúsculas (ej. /users, /products/123).
- **Representación de datos:** Uso preferente de JSON frente a formatos más pesados como XML.
- **Negociación de contenido:** Mediante cabeceras como Accept: application/json.

Modelos de Madurez de Richardson

- **Nivel 0:** POX Swamp (Plain Old XML / Uso básico de HTTP como mero túnel).
- **Nivel 1:** Uso de múltiples URIs basadas en recursos.
- **Nivel 2:** Interacción con recursos mediante los diferentes Verbos HTTP (GET, POST, etc.).
- **Nivel 3:** HATEOAS (Hypertext As The Engine Of Application State / Hipermedia como motor del estado de la aplicación).

4. SPRING BOOT Y EL ECOSISTEMA SPRING

¿Qué es?: Un framework opinionado del ecosistema Spring que simplifica la creación de aplicaciones listas para producción aplicando la filosofía de "convención sobre configuración".

Características Clave

- Aplicaciones autónomas con servidores web incrustados.
- Dependencias predefinidas y configuración automática.
- Sin generación obligatoria de código ni ficheros XML complejos.
- Integración con herramientas como Spring Initializr.

5. INYECCIÓN DE DEPENDENCIAS (DI)

Principio: Una clase declara los requisitos necesarios para operar, pero no es responsable de crear las instancias de los objetos de los que depende; estos son proporcionados desde el exterior.

Ventajas: Menos acoplamiento, mayor testabilidad y mantenibilidad. En Spring, esto se gestiona mediante el contenedor de inversión de control (IoC) y Beans.

Tipos de configuración por anotaciones

- **Estereotipos a nivel de clase:** @Component, @Service, @Repository, @Controller.
- **Inyección automática:** @Autowired (o mediante constructores de forma nativa).

6. DESARROLLO DE UN CONTROLADOR REST CON SPRING BOOT

@RestController: Anotación a nivel de clase que combina @Controller y @ResponseBody. Indica que la clase es un componente controlador cuyos métodos devuelven datos directamente serializados (por defecto a JSON mediante HttpMessageConverters como Jackson).

Mapeo de Endpoints

- **@GetMapping:** Para peticiones GET.
- **@PostMapping:** Para peticiones POST.
- **@PutMapping:** Para peticiones PUT.
- **@DeleteMapping:** Para peticiones DELETE.

Extracción de Parámetros y Datos de Entrada

- **@RequestBody:** Permite deserializar y extraer el cuerpo JSON de una petición (típico en POST o PUT) para mapearlo a un objeto Java.
- **@PathVariable:** Permite inyectar variables contenidas directamente en la ruta de la URI (ej. /product/{id}). Soporta expresiones regulares mediante PathPattern (ej. {id:[0-9]+}).
- **@RequestParam:** Permite extraer parámetros enviados en la query string de la URL (ej. ?maxPrice=23.45&sort=desc). Admite parámetros opcionales (required = false) y valores por defecto (defaultValue).

Valores de Retorno

- **Objetos POJO:** Spring los convierte automáticamente a JSON utilizando Jackson.
- **ResponseEntity<T>:** Clase envoltorio avanzada que permite controlar de forma explícita el código de estado HTTP, las cabeceras personalizadas y el cuerpo de la respuesta.

7. RESUMEN CRUD Y CORRESPONDENCIA CON MÉTODOS HTTP

Acción CRUD	Método HTTP	Ruta típica	Código Respuesta Correcta	Código Respuesta de Error
Obtener lista	GET	/product/	200 OK	404 Not Found
Obtener uno	GET	/product/{id}	200 OK	404 Not Found
Crear	POST	/product/	201 Created	400 Bad Request
Actualizar	PUT	/product/{id}	200 OK	400 Bad Request, 404 Not Found
Borrar	DELETE	/product/{id}	204 No Content	404 Not Found